

VELARIS INTELLIGENCE · SDE INTERN TECHNICAL CASE STUDY

SignalWatch

Turn messy public-event data into company risk intelligence.

Python · PySpark · REST API · NumPy · Matplotlib No frontend. No scraping.

48h

DEADLINE

6-8h

REAL EFFORT

100

POINTS

+10

BONUS

01 The Brief *Six moves, in order.*

1 Collect

Pull records from the REST API and the CSV.

2 Clean

Validate and normalise into one shape.

3 Dedupe

Find and drop repeated events.

4 Process

Transform and aggregate in PySpark.

5 Score

Company risk score + Low/Med/High.

6 Serve

REST API, NumPy stats, one chart.

02 The Data *150 records, two sources, messy on purpose.*

A single event record

```
{
  "event_id": "EVT-1024",
  "company_name": "Northstar Logistics",
  "category": "supply_chain",
  "severity": 4,
  "confidence": 0.85,
  "published_at": "2026-07-20T10:30:00Z",
  "country": "India",
  "source": "Public Business Monitor",
  "description": "Operations were
    disrupted by a transport strike."
}
```

Field rules

event_id not reliably unique
company_name required · formatting varies
category required · 6 canonical values
severity required · integer 1–5
confidence required · decimal 0–1
published_at required · 6 date formats
country optional
source optional
description dedup signal

YOUR DATA PACK

events_api.json 90 records, served by **mock_api.py** (paginated — read every page). **events.csv** 60 records. Full field notes in **DATA_DICTIONARY.md**.

EXPECT THIS, HANDLE IT, DON'T CRASH

Duplicate events · missing values · inconsistent company names · invalid severity · broken date formats · low-confidence records. **One bad record must never kill the run — log rejects or write them to a separate file, with the reason.**

03 Core Requirements *Four components. Build them in this order.*

Ingestion

- Fetch from the REST API (all pages)
- Read the CSV
- Merge into one structure
- Handle errors, timeouts, bad responses
- Log processed vs rejected counts

Validation & Normalization

- Trim whitespace, normalise company names
- Map categories to one canonical set
- Validate severity 1–5, confidence 0–1
- Parse every date format to one standard
- Reject cleanly, never crash

Deduplication

- Same company + category + description = duplicate
- Formatting differences must still match
- Pick your own strategy
- Explain the reasoning in the README

PySpark Processing

- Load cleaned records, apply an explicit schema
- Compute event-level risk scores
- Aggregate by company, classify the level
- Write JSON / CSV / Parquet output
- Pandas as the main engine does NOT count

04 Risk Scoring *Follow this exactly — it is what we test against.*

Event Risk Score = $\min(100, \text{Severity} \times 20 \times \text{Confidence} \times \text{Recency Weight})$
 Company Risk Score = average of that company's five highest event scores

REGENCY WEIGHT

0–7 days	1.0
8–30 days	0.8
More than 30 days	0.6

CLASSIFICATION

LOW 0 – 39.99	MEDIUM 40 – 69.99	HIGH 70 – 100
-------------------------	-----------------------------	-------------------------

Fewer than five events? Average all the valid events the company has.
 Round every score to 2 decimal places.

05 REST API *FastAPI preferred. Flask or DRF are fine.*

	ENDPOINT	BEHAVIOUR
GET	<code>/health</code>	Returns { "status": "healthy" }
POST	<code>/ingest</code>	Runs the full pipeline, stores results, returns the summary below
GET	<code>/companies</code>	Filters: risk_level · country · minimum_score · limit
GET	<code>/companies/{company_name}</code>	name · risk_score · risk_level · event_count · top_categories
GET	<code>/events</code>	Filter by at least two of: company · category · min score · country · date range

```

POST /ingest -> { "status": "completed", "received_records": 150, "valid_records": 132,
                  "rejected_records": 8, "duplicate_records": 10, "companies_processed": 24 }
  
```

06 Analytics, Tests & Structure *Small sections, easy points.*

NumPy – four statistics

From the final company risk scores: mean, median, standard deviation, 90th percentile.

Matplotlib – one useful chart

Recommended: Top 10 Companies by Risk Score. Needs a title, labelled axes, readable company names and visible values.

```
outputs/top_company_risks.png
```

Tests – at least three

Cover risk-score calculation, invalid-record handling, duplicate detection, API responses or company aggregation. One documented command:

```
pytest
```

Structure – your call

A suggested layout: `app/ (api · ingestion · processing · models · services · utils) · data/ · outputs/ · tests/ · requirements.txt · README.md · .gitignore`

07 Submission *Public or accessible GitHub repo, within 48 hours.*

In the repo

- ☐ Complete source code
- ☐ README.md and requirements.txt
- ☐ Automated tests
- ☐ Sample output + the generated chart
- ☐ Clear run instructions
- ☐ Meaningful Git commit history

In the README

1 Overview · 2 Architecture · 3 Setup · 4 API docs · 5 Risk-scoring explanation · 6 Deduplication strategy · 7 Assumptions · 8 Known limitations · 9 What you'd improve with more time · plus the AI tools you used.

Never commit passwords, tokens, virtual environments or generated junk.

08 Levels & Scoring *Finish Level 1 before touching anything else.*

LEVEL 1 – CORE

Ingestion · validation · PySpark · risk scores · required APIs · NumPy · Matplotlib · README · basic tests

LEVEL 2 – STRONG

Modular architecture · typed Python · detailed error handling · structured logging · pagination · env config · unit + integration tests · SQLite/Postgres/Mongo

LEVEL 3 – EXCEPTIONAL

Docker · PostgreSQL · incremental processing · background jobs · auth · caching · GitHub Actions · data-quality metrics · NLP categorisation

Functional correctness	25
Code quality	15
REST API design	15
PySpark implementation	15
Validation + dedup	10
Docs & Git	10
NumPy + Matplotlib	5
Automated testing	5
Total	100

+10 bonus for thoughtful optional work.

AI TOOLS

Use them. Name them in the README and flag any significant AI-generated components. **AI usage will not reduce your score — inability to explain the submitted work will.** Shortlisted candidates get a 30-minute discussion: walk the architecture, debug a record, change the scoring rule, add a filter, explain a commit.

A simple solution with clear reasoning beats an over-engineered one nobody can run.

We're interested in how you build, think, prioritize and communicate. Build something that works. Keep it understandable. Be ready to explain your decisions.

Build. Ship. Impact. Repeat.